



分层混合拓扑的协调复杂度：多智能体矩阵从 $O(N)$ 扇入到 $O(\log N)$ 轮次的百万级外推

UDOS 推演引擎 v7.5.0 · 房文新 · 2026-09-20

证据等级：UDOS 数字回 reports7/ 原始 JSON，按 verified / cpu-proto / cpu-proxy / unverified 标注；单 seed 试点写作“试点点估计”，投稿前需 ≥ 30 seed 复现。

本文为 UDOS 推演引擎 v7.5.0 (main 分支 tag v7.5.0, commit 1a71270) 的系统/测量型论文。全部实测数字来自本仓固定 seed 的 CPU 确定性原型, 证据等级为 `cpu-proto`; 小规模逐边显式模拟与闭式计数互验一致后, 才对更大规模做闭式外推。当前主结果为单 seed 试点点估计, 投稿前需补 ≥ 30 个独立 seed 并报告置信区间 (见第 5、7 节)。

结构化摘要 (背景与问题 → 方法 → 证据与结果 → 贡献)

【背景与问题】 多智能体系统从十几个 Agent 扩到数十万乃至上百万时, “用多少 Agent” 不再是唯一瓶颈, “Agent 之间如何协调” 成为能否收敛的关键; 中心节点扇入爆炸、关键路径串行化、故障在网状结构中放大, 是三个尚未被同口径测量的环节 (详见 §1.1、§3.1)。

【方法】 以零网络、零大模型调用的确定性离散事件模拟器为实验台, 把协调复杂度操作化为三件套——关键节点扇入、关键路径轮次、故障下收口率; 对星型 Orchestrator、链式 Handoff、网状 Swarm 与本文提出的 b 叉分层混合树 (b=8、每 worker 满载 4 单元) 跑同一套处理器, 把”拓扑效应”与”能力效应”解耦; 并在实际节点 $\leq 4,000$ 档用逐边显式模拟 `ExplicitSim` 守护闭式计数 `route()` (详见 §4)。

【证据与结果】 在 10 到 1,000,000 共 6 个规模档: 星型中心扇入随 N 线性增长 (百万档中心承接 20,971,520 条消息、串行 8,388,608 轮), b 叉分层树使每节点最大扇入恒为 $b+1=9$ 、并行轮次仅 $2 \cdot \log_b N$ (百万档 14 轮); 小规模逐边显式模拟与闭式计数逐值互验误差为 0 ($400=400$ 、 $3,216=3,216$), 更大规模以 `grade` 字段强制区分 `explicit/analytical`; 故障端到端 (24 工单、9 specialist、7 QA 含 2 拜占庭、3 故障 specialist) 实现 24/24 收口、3 故障节点全隔离、5 次返工、治理与哈希链 Trace 完整 (详见 §6)。

【贡献】 (i) 把”扇入上界 + 关键路径轮次 + 故障收口率”三件套用同一显式模拟器互验; (ii) 给出可证伪闭式命题 (H1/H2/H3) 并以显式-闭式逐值互验为证伪判据; (iii) 诚实分级的百万级外推, 逐档标注证据等级与外推边界。全部数字为 `cpu-proto`、单 seed 试点, 不与任何外部厂商规模数字互证 (详见 §7、§8)。

中文摘要

随着多智能体 (multi-agent) 系统从十几个 Agent 扩展到数十万乃至上百万个协作单元, “用多少 Agent” 不再是唯一瓶颈, “Agent 之间如何协调” 成为决定系统能否收敛的关键。本文以一个零网络、零大模型调用的确定性离散事件模拟器为实验台, 系统测量三种拓扑——星型 Orchestrator、链式 Handoff、网状 Swarm——以及本文提出的 b 叉分层混合树 (顶层 Orchestrator / 中层 Handoff / 底层 Swarm) 在不同规模下的协调消息数、关键节点扇入与关键路径轮次。我们以 b=8、每 worker 满载 4 个工单单元, 在 10 到 1,000,000 共 6 个规模档上得到如下结论: 星型中心扇入随单元数线性增长 (百万档中心承接 20,971,520 条消息、串行 8,388,608 轮), 而 b 叉分层树使每个节点最大扇入恒为 $b+1=9$ 、并行轮次为 $2 \cdot \log_b N$ (百万档仅 14 轮)。小规模 (实际节点 $\leq 4,000$) 逐边显式模拟与闭式计数逐值互验, 误差为 0; 更大规模仅使用已被显式验证的闭式公式外推, 并在报告中以 `grade` 字段强制区分 `explicit` 与 `analytical`。在故障注入端到端任务中 (24 工单、9 名 specialist、7 名 QA 含 2 名拜占庭 QA、3 个故障 specialist), 分层混合拓扑配合熔断改派实现 24/24 收口、3 个故障节点全部隔离、5 次返工, 治理审计与哈希链 Trace 完整。本文贡献在于: 把”扇入上界 + 关键路径轮次 + 故障收口率”三件套用同一显式模拟器互验, 并诚实标注每一

档证据等级与外推边界。我们强调，所有结论均为纯 CPU 确定性原型下的协议层口径，不是真实墙钟吞吐，也不与任何外部厂商的规模数字互证；当前主结果为单 seed 试点，投稿前需补多种子。

关键词：多智能体系统；协调复杂度；层次化拓扑；扇入；可扩展性；故障容错；确定性仿真

English Abstract

As multi-agent systems scale from a dozen agents to hundreds of thousands or millions of cooperating units, “how many agents” ceases to be the only bottleneck; “how agents coordinate” becomes decisive. Using a deterministic discrete-event simulator with no network and no large-model calls as the testbed, this paper measures, across three topologies (star Orchestrator, chain Handoff, mesh Swarm) and the proposed b-ary layered hybrid tree (top Orchestrator / middle Handoff / bottom Swarm), the coordination message count, critical-node fan-in, and critical-path rounds at each scale. With branch factor $b=8$ and four work units per worker, across six scale tiers from 10 to 1,000,000, we find that star center fan-in grows linearly with unit count (at the million tier the center handles 20,971,520 messages over 8,388,608 serial rounds), whereas the b-ary layered tree keeps every node’s maximum fan-in constant at $b+1=9$ and parallel rounds at $2 \cdot \log_b N$ (only 14 rounds at the million tier). At small scale ($\leq 4,000$ realized nodes), edge-by-edge explicit simulation matches closed-form counts exactly (error 0); larger tiers use only the already-verified closed form, with a `grade` field explicitly separating `explicit` from `analytical`. Under fault-injected end-to-end missions (24 work orders, 9 specialists, 7 QA voters including 2 Byzantine, 3 faulty specialists), the layered hybrid topology with circuit-breaker reassignment achieves 24/24 completion, isolates all 3 faulty nodes, incurs 5 reworks, and keeps governance audit and hash-chained Trace intact. Our contribution is to cross-validate a fan-in upper bound, critical-path rounds, and fault-completion rate within one explicit simulator, while honestly labeling the evidence grade and extrapolation boundary of every tier.

Keywords: multi-agent systems; coordination complexity; hierarchical topology; fan-in; scalability; fault tolerance; deterministic simulation

1 引言

1.1 外部产业动机（外部数字均为 `unverified`，报道口径，未独立核验）

2025–2026 年，“Agent 军团 / Agent 编排”成为产业界高频叙事。多家厂商与研究机构宣称其系统可同时调度成百上千乃至上万个子 Agent 完成软件工程、数据分析、研究助理等任务；外部报道中出现了“万级 Agent 并行”“多智能体协作平台”等说法（此类外部数字属报道口径，未独立核验，本文标记为 `unverified`，仅作动机，不与本仓任何数字互证）。[CITATION NEEDED: multi-agent orchestration platform at scale, agent swarm benchmark]

这些叙事共同的隐含假设是：只要把更多 Agent 塞进系统，吞吐与质量就会线性提升。但分布式系统的历史反复表明，协调开销（coordination overhead）往往先于计算资源成为瓶颈：中心节点扇入爆炸、关键路

径串行化、故障在网状结构中放大。本文关心的不是”能不能跑起来”，而是”在什么拓扑下，协调开销随规模的增长是可证明可控的”。

需要强调：本文不引用、不背书任何外部厂商的性能数字。外部叙事只用于说明”为什么这个问题值得在可复现的实验台上被严格测量”。

值得进一步指出，外部叙事往往只报告”成功案例”：某个演示中 N 个 Agent 协同完成了某任务。但在系统研究看来，演示成功与”可扩展”之间隔着三个尚未被测量的环节：其一，**中心节点的瞬时压力**——如果所有结果都汇总到一个调度器，它的扇入是否随 N 线性增长；其二，**关键路径长度**——从任务下达到最终收口，是否随 N 串行拉长；其三，**故障放大**——当少量 Agent 出错或给出拜占庭式错误输出时，网状结构是把故障局部化，还是让它污染全局。本文正是要在一个可复现实验台上，把这三个环节变成可测量、可证明的量。

1.2 从”堆人”到”排结构”：一个被忽视的转折

把多智能体系统类比为人类组织是有帮助的。一个十人的创业团队可以围着一张桌子直接沟通；但当组织扩张到上万人时，没有人会让每个人都直接向 CEO 汇报——那样 CEO 的收件箱会在一天内爆炸，且任何一条迟到的汇报都会拖慢整体决策。人类组织用部门、层级、汇报线来把”扇入”控制在每个管理者可承受的范围内。多智能体系统在工程上正经历同一个转折：早期的十几个 Agent 可以自由对话，但当 Agent 数量向百万级演进时，”让多少 Agent 直接和中心说话””任务结果要经过多少层才能汇总到决策点”，就从工程细节变成了决定系统能否收敛的结构性问题。

本文要回答的不是”多智能体能不能协作”，而是更冷硬的问题：在不同拓扑下，协调开销随 Agent 数量的增长阶是什么？哪一种结构能让单节点扇入与规模解耦，又让关键路径随对数而非线性增长？这个问题之所以值得做，是因为它在多智能体文献中常被端到端成功率或单轮对话质量所掩盖，却恰恰是决定”能不能上量”的底层约束。

我们把”协调复杂度”操作化为三个可测量、可证明的量：

1. **关键节点扇入 (fan-in)**：在一轮中，某个节点需要同时接收多少条消息。扇入决定了该节点的瞬时处理压力与单点故障影响面。
2. **关键路径轮次 (critical-path rounds)**：从任务下发到结果收口，必须经过多少个串行轮次。轮次决定了端到端延迟的下界。
3. **故障下收口率 (completion rate under faults)**：在 crash / drop_context / byzantine 等故障注入下，有多少比例的工单最终被正确验收收口。

我们的核心论点是：**拓扑结构决定了这三个量随规模 N 的增长阶**。星型拓扑把扇入和轮次都压在中心节点上，随 N 线性增长；而 b 叉分层聚合树通过把协调责任沿层级分摊，使单节点扇入与 N 无关、轮次随 $\log_b N$ 增长。

1.3 贡献

本文的贡献如下：

- **可证伪的形式化命题**：给出星型与 b 叉分层树在扇入、轮次上的闭式表达式 ($H1$ 、 $H2$)，并把”显式模拟与闭式计数是否逐值一致”作为可证伪判据。这意味着结论不是”我们观察到分层更快”，而是”如果显式模拟测出的扇入与 $b+1$ 不符，我们的命题即被推翻”——把结论放在可被实验否定的位置上。

- **显式-闭式互验实验台**：在 `udos7/topology/hierarchy.py` 中实现 `ExplicitSim`（逐边模拟）与 `LayeredMatrix.route()`（闭式计数），在实际节点 $\leq 4,000$ 的两档上逐值比对，误差为 0（ $400=400$ 、 $3,216=3,216$ ）。
- **诚实分级的百万级外推**：6 个规模档中前两档标 `explicit`、后四档标 `analytical`，报告以 `grade` 字段强制区分，绝不把闭式外推冒充成实测。这是本研究在方法论上区别于”直接外推到百万级”叙事的核心。
- **故障端到端收口证据**：在 `udos7/topology/matrix.py` 中把黑板派单、分层 `specialist`、熔断改派、BFT-lite QA、内部市场、哈希链 Trace 串成闭环，故障注入下 24/24 收口、3 故障节点全隔离。
- **证据纪律**：所有数字标注证据等级与来源文件，当前单 seed 的局限被显式写入正文，不做过度声称。

1.4 范围声明与读者对象

本文的读者是做大规模多智能体系统架构、分布式协调协议、以及关心”可复现测量”的系统研究者。本文不研究学习算法本身（Agent 的智能由外部模型提供，本文将其抽象为确定性处理器），不研究真实网络协议栈，也不承诺生产 SLA。我们刻意把研究边界划在”协调协议的复杂度与故障收口”上，以便在纯 CPU、可复现的条件下得到确定结论。超出这一边界的结论（如真实墙钟延迟、真实 LLM 故障分布）在第 8 节闸门中显式标注为未达成。

1.5 论文结构

第 2 节综述相关工作；第 3 节给出问题定义与可证伪假设；第 4 节描述系统设计与伪代码；第 5 节给出实验设置；第 6 节报告结果；第 7 节讨论与威胁；第 8 节资源闸门与适用边界；第 9 节结论；末尾为参考文献、附录与内审记录。

2 相关工作

2.1 层次化多智能体与群体智能

层次化组织结构在人类组织与分布式计算中由来已久。**概念**上，任务需要在”能者居之”的原则下被分发出去；**机制**上，Smith 的合同网协议（Contract Net Protocol）用”招标—投标—授标”三步让任务在节点间动态竞价，是任务分配与拓扑自组织的源头经典（Smith, 1980, IEEE Trans. Computers C-29(12), DOI: 10.1109/TC.1980.1675516）。**证据**上，合同网证明了”显式协商”能在无中心地图的情况下把任务派给合适节点，但其代价是每任务多轮广播与投标消息。多智能体强化学习与任务分配后续讨论过”分层调度—局部协作”的混合结构，以期在可扩展性与局部自治之间取得平衡；这类研究通常把上层作为价值分配或策略蒸馏的中枢，下层作为局部执行，关注点多在学习收敛而非消息扇入。**与 UDOS 的差异**在于：UDOS 不把协商当作学习过程，而是把合同网式的广播-投标-授标作为一条**被测基线**（`udos7/topology/matrix.py` 中的 `contract-net` 分支），与 b 叉分层树放到同一把尺子下测量扇入——即我们关心的不是”能不能分配”，而是”分配这条消息路径的扇入上界是多少”。

群体智能（swarm intelligence）则强调去中心化、通过环境媒介（stigmergy）间接协调，避免中心扇入：**概念**上，协作信号不必点对点直达，而可编码到共享环境上，个体只读取局部痕迹；**机制**上，Dorigo 等

人的蚁群系统 (Ant System) 用信息素随路径留下、随时间蒸发, 让大量简单个体仅凭局部痕迹完成最短路径寻优 (Dorigo, Maniezzo & Colorni, 1996, IEEE Trans. SMC-B 26(1))。证据上, 蚁群证明了”间接通信”能在无全局地图下涌现出协同行为。本仓 `udos7/topology/stigmergy.py` 实现了这一思路的确定性版本——Agent 之间不直接协商, 而是在共享黑板 `Blackboard` 上原子认领任务、留下完成标记, 信息素随时间蒸发引导后续调度。在 30 任务/5 Agent 的小基准上 (`tests7/test_v7410_stigmergy.py::test_stigmergy_uses_fewer_messages_than_contract_net`, 投票者数 `n_bidders=5`), `stigmergy` 每任务 2 条痕迹共 60 条消息, 而 `contract-net` 广播-投标-授标每任务 8 条协商消息共 240 条; 在 40 任务/4 Agent 负载均衡基准上 (`test_load_balanced_across_agents`) 各 Agent 负载差 ≤ 1 。与 UDOS 的差异在于: 蚁群的信息素是启发式寻优信号, UDOS 的黑板痕迹是确定性的任务认领记录; 二者共享”用环境媒介替代点对点协商”的机制, 但 UDOS 把这一机制的消息开销纳入显式计数, 使”减少直接协商流量”从定性直觉变成可测数字。

本文不主张某一种拓扑绝对最优, 而是把 Orchestrator (中心)、Handoff (链式接力)、Swarm (网状协商) 以及本文提出的 b 叉分层混合树, 放到同一把尺子下测量扇入与轮次。三种纯拓扑在 `udos7/topology/base.py` 中跑同一套处理器 (`trriage_handler/specialist_handler/qa_handler`), 差异只在控制权与消息路径, 从而把”拓扑效应”与”能力效应”解耦——这是测量可信度的关键设计。

2.2 扇入、可扩展性与聚合树

在分布式计算中, 树型聚合 (tree aggregation) 的消息复杂度与关键路径长度是经典结果: k 元树的聚合深度为 $\log_k N$, 每节点子节点数为 k 。MapReduce、serverless fan-out 与各类广播/归约协议都利用了这一结构。[CITATION NEEDED: tree-based aggregation message complexity, broadcast reduce critical path] 与”并行能走多快”同源的另一条经典线索是 Amdahl 定律: 概念上, 无论并行度多高, 系统速度受限于必须串行执行的那一小部分; 机制上, Amdahl 给出加速比 $S(N)=1/(s+(1-s)/N)$, 其中 s 为串行比例; 证据上, 这条定律从 1967 年起就是衡量任何并行扩展上限的标尺 (Amdahl, 1967, AFIPS SJCC 1967)。与 UDOS 的差异在于: 经典树聚合与 Amdahl 都假设”叶子产生数据、内部节点只做求和”或”任务可任意并行”, 其消息模型较简单; 而多智能体协调中, 每个节点都可能是有状态的决策者, 扇入不仅是”数据到达数”, 还包括”派发下行 + 汇报上行 + 聚合”三类消息。本文的闭式模型把这三类分别计数 (下行边、任务下发、worker 汇报、逐级聚合), 并用 `ExplicitSim` 实测收件箱, 正是为了在更贴近多智能体语义的消息模型下重新验证这个经典阶——而 b 叉分层树之所以把中心扇入从线性压到常数 9, 本质上就是用树深度换取了单节点扇入, 这正是 Amdahl 式”用结构换瓶颈”在协调扇入维度上的体现。

这一区分值得强调: 经典树聚合通常假设”叶子产生数据、内部节点只做求和”, 其消息模型较简单; 而多智能体协调中, 每个节点都可能是有状态的决策者, 扇入不仅是”数据到达数”, 还包括”派发下行 + 汇报上行 + 聚合”三类消息。本文的闭式模型把这三类分别计数 (下行边、任务下发、worker 汇报、逐级聚合), 并用 `ExplicitSim` 实测收件箱, 正是为了在更贴近多智能体语义的消息模型下重新验证这个经典阶。

2.3 故障容错与拜占庭容错

熔断 (circuit breaker) 在微服务领域成熟, 断路器的 closed/open/half-open 状态机、冷却探测与半开恢复是通用模式。[CITATION NEEDED: circuit breaker pattern microservices] 拜占庭容错 (BFT) 共识是另一条成熟机制: 概念上, 异步网络中存在可能任意作恶 (equivocation) 的节点时, 如何仍就状态达成一致; 机制上, Castro 与 Liskov 的实用拜占庭容错 (PBFT) 用 pre-prepare/prepare/commit 三阶段状态机复制, 要求 $n \geq 3f+1$ 才能容忍 f 个拜占庭节点, 并需 $2f+1$ 法定人数, 配合 view change 处理主节点失效; 证据上, 其实现的 BFS 文件系统仅比未复制 NFS 慢约 3% (Castro & Liskov, 1999, OSDI 1999)。

PBFT 的代价是 $O(n^2)$ 消息复杂度、视图更换昂贵，不适合大规模；Yin 等人的 HotStuff 进一步把 BFT 做到线性通信复杂度且响应式，用 leader 主导、链式投票、模块化视图更换（Yin, Malkhi, Reiter et al., 2019, PODC 2019, arXiv:1803.05069）。与 UDOS 的差异在于：BFT/HotStuff 面向分布式状态机复制，关注一致性；UDOS 只把其中的 $n \geq 3f+1$ 、 $2f+1$ 法定人数与 equivocation 检测当作“QA 委员会投票”这一局部构件（详见 P3），用它来对“验收通过/返工”投票，而非跑完整共识。本文把这些机制作为现成构件集成进多智能体矩阵，并在故障注入下测量其对收口率的因果效应。

在本系统中，熔断分三级：Agent 级滑动窗口错误率超阈值即 open（隔离），冷却后半开探测、成功才恢复 closed；子矩阵级隔离比例超阈值则整组熔断、任务改派健康组；全局 kill-switch 一旦置位则停止一切派发。回滚机制在派发前对哈希链 Trace 做快照（记录长度与末哈希），故障后截断半截状态并重派，保证不留半截状态。BFT-lite QA 委员会对 stop/continue 投票， $n \geq 3f+1$ 、 $2f+1$ 法定人数，拜占庭谎报无法对抗诚实多数。这些构件不是本文的新颖性重点，但它们是“分层树在故障下仍收口”得以成立的工程基础。

2.4 现有工作的缺口

我们假设（待系统检索证伪/证实）：现有多智能体框架文献多报告端到端时延或任务成功率。概念上，以 AutoGen 为代表的可对话多智能体框架证明了 LLM/人/工具混合多智能体在数学、代码、运筹等多领域可搭应用（Wu, Bansal, Zhang et al., 2023, arXiv:2308.08155）；机制上，它用可定制、可对话的 agent 编排协作；证据上，其局限是“对话无终止条件易死循环、成本随轮数膨胀”，即框架关心对话能否完成，而非扇入与轮次的上界。与 UDOS 的差异在于：AutoGen 类框架较少把“消息扇入上界 + 关键路径轮次 + 故障收口率”三件套用同一显式模拟器互验，并在报告中按 `explicit / analytical` 分级外推到百万级。[CITATION NEEDED: multi-agent framework scalability measurement fan-in critical path] 这一缺口假设本身需要在投稿前通过学术检索确认，若已存在同口径扇入上界证明，本文将退化为“复现并扩展到多智能体协调语义 + 故障收口”的增量贡献。

2.5 测量方法论与可复现传统

系统测量研究的可信度来自两个习惯：一是用最小可复现实验台隔离变量，二是把“实测”与“外推”在报告中分开标注。[CITATION NEEDED: empirical systems measurement reproducibility methodology] 分布式系统社区有“先在小规模上互验模型、再外推到大集群”的传统（如用排队论或离散事件仿真先预测、再在生产验证）。本文延续这一传统：`ExplicitSim` 相当于离散事件仿真，`route()` 相当于解析模型，二者在可达规模上逐值互验后才外推。区别在于，我们把“哪里是仿真、哪里是解析模型”固化进数据 schema（`grade` 字段），而不是留给读者自行判断。这一做法与本仓 P12 论文讨论的证据分级方法论一脉相承。

2.6 与范本范式的对照（NSA 稀疏化 / AlexNet 消融）

本文的设计与三篇逆向拆解范本（见 A0 报告）形成方法论对照，此处点明两处，便于审稿定位：

- 对照 NSA”硬件对齐 → 稀疏化”：NSA（DeepSeek, 2502.11089）从 decode 阶段的内存墙（memory-bound）出发，把“理论加速 ≠ 实际延迟”这一幻象钉死，再用 blockwise 连续访问的稀疏化把算术强度对齐到硬件。本文面对的是一个对偶的“扇入墙”：星型 Orchestrator 的中心节点瞬时扇入 $2U + 2W$ 随 N 线性爆炸（百万档 20,971,520），这是协调层的“单点内存/处理墙”。本文的对应解法不是“更硬地堆中心算力”，而是沿层级做扇入稀疏化—— b 叉聚合树把每个节点的瞬时扇入截断为常数 $b + 1 = 9$ ，与 NSA”把全量注意力稀疏化为 blockwise 选择”在结

构上同构：二者都是”先识别墙在哪一环，再用结构化稀疏把该环的复杂度从线性压到常数/对数”。差别在于 NSA 稀疏的是 KV 激活 token，本文稀疏的是汇聚扇入；NSA 以同 token 预算对照 Full Attention，本文以同处理器对照星型。

- **对照 AlexNet” 每个 trick 配数值消融”**：AlexNet 把双 GPU 分裂、LRN、重叠池化各自配”移除后掉多少误差”的数值消融。本文对”分层是否真的必要”给出对偶证据：§ 6.8 用分支因子 b 的公式敏感性 (b=2/8/16 下扇入 3/9/17、轮次约 40/14/10) 刻画”把扇入换轮次”的旋钮；§ 6.2 用显式-闭式逐值互验 (误差 0) 证明”分层树的扇入不是约定出来的，而是真跑出来的”。这正是 AlexNet 范式在系统测量中的移植：不喊”分层更好”，而是给出”移除/改变结构后，哪个可观测测量变了多少”。

3 问题定义与可证伪假设

3.1 系统模型

设系统有 N 个协作 Agent。每个工单 (work order) 需要在三阶段 (triage → specialist → qa) 上被处理，成功可机械判定 (结果等于工单内真值且领域匹配)。协调动作以”消息/轮次”为单位计数：一轮中，每个节点可向一个目的地发送一条消息，也可接收来自多个来源的消息。我们不模拟真实网络延迟、不模拟真实 LLM 推理，因此所得结论是**协议层协调复杂度的上界/口径**，不是墙钟吞吐 (见第 7 节构念效度)。

3.2 假设 H1 (扇入)

设 U 为满载时下发的工单单元数，W 为 worker 数。

- **星型**：所有派发与汇总都经中心节点。中心节点扇入 = $2U + 2W$ (派发 + 回传)，随 U、W 线性增长：

$$F_{\text{star}} = 2U + 2W \propto N$$

- **b 叉分层树**：每个内部节点接收 b 条子级汇总 + 1 条父级派发，故最大扇入 = $b + 1$ ，与 N 无关；顶层扇入 = b：

$$F_{\text{layered}}^{\text{max}} = b + 1, \quad F_{\text{layered}}^{\text{top}} = b$$

可证伪判据：若逐边显式模拟测得的关键节点扇入与上述闭式表达式在任意规模档上不一致 (超出计数误差)，H1 被推翻。

3.3 假设 H2 (轮次)

- **星型**：中心串行处理每个单元，关键路径轮次 = U。
- **b 叉分层树**：下行派发 d 层 + 上行聚合 d 层，关键路径轮次 = $2d$ ，其中 $d = \lceil \log_b N \rceil$ 。

$$R_{\text{star}} = U, \quad R_{\text{layered}} = 2d = 2\lceil \log_b N \rceil$$

可证伪判据：若显式模拟测得的串行/并行轮次与 U、 $2d$ 不符，H2 被推翻。

3.4 假设 H3（故障下收口）

分层混合拓扑在 crash / drop_context / byzantine 注入下，配合熔断改派与 BFT-lite QA 共识，仍能 100% 收口；而纯星型在中心节点故障时收口率坍塌。

可证伪判据：若分层在非中心故障场景下收口率显著低于星型，H3 被削弱；若熔断改派后仍存在未收口工单，H3 被推翻。需要诚实说明：本文当前只测量了”分层混合拓扑 + 全量治理”这一种配置的故障收口，未在同一实验台上测量”纯星型 + 中心故障”的对照，该对照属于计划中的多 seed 因子实验（见第 8 节）。

3.5 单位与记号约定

为避免歧义，本文约定：N = 目标 Agent 规模档；b = 分支因子（本文固定 b=8）；d = 树深；W = worker 数 = b^d ；U = 满载工单单元数 = $W \times \text{units_per_worker}$ （本文 $\text{units_per_worker}=4$ ）。所有规模数字以 reports7/matrix_scale_demo.json 为唯一权威来源。

3.6 与负载均衡、排队论的关系

需要澄清一个常见误解：分层树降低的是协调开销的增长阶，不等于”负载一定均衡”。扇入恒定说的是每个节点瞬时接收的消息数有上界，而不是说任务会自动均匀分布到所有 worker。在本系统中，负载均衡由 stigmergy 黑板的原子认领与信息素加权实现（40 任务/4 Agent 负载差 ≤ 1 ，见 test_v7410_stigmergy.py::test_load_balanced_across_agents），与分层聚合树是两个正交的机制。同样，轮次是依赖图的关键路径长度，不是排队论中的平均等待时间——我们没有引入到达过程与服务时间分布，因此不做排队论式的利用率声明。把扇入、轮次与真实排队延迟联系起来，需要在真实分布式环境中测量，这属于第 8 节闸门。

4 方法与系统设计

4.1 b 叉分层聚合树

层级角色映射（源码 udos7/topology/hierarchy.py 的 LayeredMatrix.level_role）：

- level 0: Orchestrator（战略调度）；
- level $\geq d-1$: Swarm team + worker（大规模并行探索）；
- 中间层: Handoff（按领域/部门接力聚合）。

节点数闭式（源码 LayeredMatrix）：

- worker 数: $W = b^d$ ；
- 内部节点数: $n_{\text{internal}} = (b^d - 1)/(b - 1)$ （等比级数 $1+b+\dots+b^{(d-1)}$ ）；
- 总节点数: $N = W + n_{\text{internal}}$ 。

满载路由（源码 LayeredMatrix.route(n_units)）：下行边 = N-1（每个非根节点收 1 条派发）；上行边 = $n_{\text{internal}}-1$ （内部节点除根外逐级聚合）；总消息 = down + up；每节点最大扇入 = b+1；并行轮次 = 2d。

4.2 星型路由对照

源码 `star_routing(n_workers, n_units)`：总消息 = $2 \cdot n_units + 2 \cdot n_workers$ ；中心扇入同值；并行轮次 = n_units （中心串行）。

4.3 逐边显式模拟器 ExplicitSim

为避免”公式看起来对、实际边没走对”，我们实现了逐边模拟器 `ExplicitSim`。它按 (level, index) 枚举每个节点，显式发送：下行每条树边一次、任务到活跃 worker、活跃 worker 向父节点汇报、内部逐级聚合。返回实际发送总消息、各节点收件箱计数（即真实扇入）。源码中的测试断言

`explicit_matches_formula` 要求显式测得的总消息、最大扇入、顶层扇入三者同时等于闭式值。

伪代码如下：

```
函数 route_closed(b, d, n_units):          # 闭式
    W = b^d; n_internal = (W-1)/(b-1); N = W + n_internal
    down_edges = N - 1; up_edges = n_internal - 1
    down = down_edges + n_units
    up    = min(W, n_units) + up_edges
    return total = down + up, max_fanin = b+1, top_fanin = b, rounds = 2d

函数 simulate_explicit(b, d, n_units):      # 逐边
    inbox = {}; msgs = 0
    for l in 1..d: for i in 0..b^l-1: send(node(l,i))          # 下行树边
    for u in 0..n_units-1: send(worker(d, u mod W))            # 任务下发
    for w in 0..min(W,n_units)-1: send(parent(d-1, w//b))      # worker 汇报
    for l in d-1..1: for i in 0..b^l-1: send(parent(l-1, i//b))# 逐级聚合
    return total=msgs, max_fanin=max(inbox), top_fanin=inbox[root]

断言: simulate_explicit == route_closed    # 小规模逐值互验
```

4.4 证据分级外推协议

源码 `scale_benchmark(sizes, branch=8, units_per_worker=4, explicit_limit=4000)` 对每个规模档：先由 `fit_depth` 定 d，构造树，算闭式；若 $N \leq \text{explicit_limit}$ (4,000)，则额外跑 `ExplicitSim` 并写 `explicit_total_messages`、`formula_total_messages`、`explicit_matches_formula`；否则该档 `grade="analytical"`，只报闭式值。这一设计把”哪里是实测、哪里是外推”固化进数据 schema，而不是靠写作时的自觉。

4.5 端到端故障闭环（matrix.py）

百万级规模下，我们不真的启动百万个进程；而是在小规模上跑完整任务闭环

（`udos7/topology/matrix.py` 的 `run_mission`），验证”分层 + 治理”在故障下能收口，再把协调复杂度结论外推。闭环流程为：

1. Stigmergy 黑板原子认领（`Blackboard.claim`，杜绝同一任务被两人认领）；
2. 分层 specialist 执行（熔断节点被 `ResilienceGrid.healthy_agents` 排除）；
3. 失败时 `snapshot(trace)` → 执行 → 若失败 `rollback` 截断半截 Trace 并释放改派；

4. BFT-lite QA 委员会就 stop/continue 投票 ($n \geq 3f+1$ 、 $2f+1$ 法定人数)；
5. 内部市场按验收结算 (ContributionLedger, 守恒断言)；
6. 哈希链 TraceLedger + 治理审计 (audit_run)。

配置 (源码 MatrixConfig) : specialists_per_domain=3 (共 9 名 specialist), qa_voters=7, qa_f=2 (2 名拜占庭 QA), reward_per_order=10, max_rounds=20, breaker_min_requests=2, breaker_threshold=0.5, global_retry_budget=999。

4.6 为什么选 b=8：分支因子的权衡

分支因子 b 不是随意取的。从闭式表达式可以直接看出它的双重角色：一方面，分层树最大扇入 = b+1, b 越大，单节点瞬时承接的消息越多；另一方面，树深 $d = \lceil \log_b N \rceil$, b 越大，轮次 2d 越小。也就是说，b 是”单节点扇入”与”关键路径轮次”之间的调节旋钮：b 小则扇入低但轮次多，b 大则轮次少但扇入高。b=8 是本文选取的折中——百万档下扇入仅 9 (每个内部节点最多收 9 条消息)，而轮次被压到 14。若 b=2, 百万档轮次会升到约 40；若 b=16, 轮次降到约 10 但扇入升到 17。本文固定 b=8 是为了在所有规模档上保持可比，不逐档调参 (这也是预注册纪律：不得在看到结果后改 b)。

4.7 何时建议分层：拓扑决策树

分层不是默认答案。udos7/topology/decision.py 的 choose_topology / choose_layered 给出了一个朴素但明确的选型逻辑：流程明确 → 中心 Orchestrator (状态统一、责任唯一、可审计)；需要多领域专家接力 → Handoff 链式交接；能力可封装 → Agent-as-Tool；开放探索且低风险 → Swarm 去中心化协商；开放探索但高风险 → 不放任自治，先加 Orchestrator 护栏/熔断再放开 Swarm。只有当规模足够大 ($scale \geq 1000$) 且任务属于开放探索或专家接力时，才建议分层混合 (顶层 Orchestrator / 中层 Handoff / 底层 Swarm)。这一决策树把”分层树”定位为大规模开放任务的手段，而非一切任务的默认——与本文”拓扑随需求与规模选型”的主张一致。

4.8 三纯拓扑的消息代价对照 (来自 base.py)

分层混合树不是凭空设计的，它把三种已实现的纯拓扑作为层级角色。理解三者的消息代价，才能理解为什么分层要把它们组合起来。在 udos7/topology/base.py 中，三种拓扑跑同一套处理器，消息计数规则如下：

- **Orchestrator (星型)**：中心对每阶段选 Agent、派发、收回、合并；每个阶段计 2 条消息 (派发 + 回传)，worker 之间不通信。优点是状态集中、责任唯一，缺点是中心扇入与轮次随规模线性增长 (即表 1 的星型列)。
- **Handoff (链式)**：工单与上下文沿 triage→specialist→qa 接力，责任随 Transfer Bundle 转移；每个阶段计 1 条移交消息。优点是上下文随责任流动、可审计，缺点是链路串行、单点交接失败会中断整条链 (drop_context 故障正对应此)。
- **Swarm (网状)**：每阶段广播任务，有能力者投标，按负载/确定性规则中标；消息数 = 广播 + 投标 ($\text{len}(\text{bidders})$) + 授标 + 结果回公告板。优点是去中心化、可负载均衡，缺点是协商流量随投标者数线性增长 (contract-net 每任务约 8 条协商消息)。

分层混合树的设计意图正是取长补短：顶层用 Orchestrator 的集中可审计，中层用 Handoff 的责任接力，底层用 Swarm 的并行探索与黑板认领。扇入与轮次的可观测量（表 1）证明这种组合确实把星型的 $O(N)$ 中心扇入解耦了。

5 实验设置

5.1 数据与配置

- 唯一数据源： `reports7/matrix_scale_demo.json` （ `evidence_grade: cpu-proto` ），生成脚本 `scripts7/matrix_demo.py` 。
- 分支因子 $b=8$ ；每 worker 满载 4 单元；显式上限 4,000 节点。
- 规模档：目标 Agent 数 $\in \{10, 100, 1,000, 10,000, 100,000, 1,000,000\}$ 。
- 端到端任务：24 工单（ `standard_workload(24, seed=11)` ），干净跑 `seed=1`；故障注入跑 `seed=1`。

5.2 硬件指纹与随机种子

- 运行环境：Linux CPU；Python 3.12.11；PyTorch 2.14.0+cpu；2 线程；device=cpu（硬件指纹与 `reports7/v7_verification.json`、`reports7/agent_scaling.json` 记录一致）。
- 随机种子：端到端 mission `seed=1`，工单 `seed=11`；规模基准为确定性计数，无种子。
- 报告 sha256： `reports7/matrix_scale_demo.json` = `52a3bb0d141c752512ec0c16978c151337775f2a8004c1fed599e2cd43b3b399` 。

5.3 证据等级标注约定

- `verified`：本仓固定 seed 可复跑的实测（如小规模显式-闭式互验断言）。
- `cpu-proto`：CPU 确定性原型/模拟，Agent 非真实 LLM、无真实网络/GPU（本文主体）。
- `analytical`：由已被显式验证的闭式公式外推，非逐边实测。
- `unverified`：外部报道/厂商口径，仅进引言与讨论。

5.4 单 seed 局限与统计计划

当前端到端为单 seed（`seed=1`）试点点估计。规模计数为确定性，无 seed 间方差；但故障注入的收口率、返工次数存在 seed 间方差。投稿前需补 ≥ 30 个独立 seed，报告均值与 95% bootstrap 置信区间；效应量与功效分析在补 seed 后填入 [RESULT NEEDED: 多 seed 收口率均值与 95% CI、 η^2 、功效]。本文不虚构任何 p 值、CI 或样本量。

5.5 测量口径与计数规则

为保证可复现，本节明确”一条消息”和”一轮”的定义。在闭式模型中：每个非根节点在树建立时收一条父级派发（计 1 条下行边）；满载时每个 worker 收一条任务下发；每个活跃 worker 向上级汇报一次；每个非根内部节点向其父级聚合一次。扇入 = 某节点收件箱累计收到的消息数（由 `ExplicitSim.inbox` 直接

计数，而非事后推算）。轮次按依赖关系分层计算：下行 d 层、上行 d 层，故并行轮次 = 2d；星型因中心必须逐个处理单元，轮次 = U。

这一口径的关键是：扇入由逐边模拟实测收件箱得出，而不是把 b+1 当作定义硬写进去。只有当显式模拟测出的最大收件箱计数等于 b+1 时，才说明”扇入恒定”是真的跑出来的，而不是约定出来的。测试 `explicit_matches_formula` 同时校验总消息、最大扇入、顶层扇入三者，正是为了防止”总消息对了但扇入其实不对”这类局部正确。

5.6 基线、对照与预注册冻结项

本文以星型 Orchestrator 为基线（对照），分层混合树为处理组。为避免”看到结果后调参”，以下配置在收集数据前冻结：分支因子 b=8、每 worker 满载单元数 units_per_worker=4、显式上限 explicit_limit=4000、规模档集合 {10,100,1000,100000,1000000}。故障端到端的配置也冻结：specialists_per_domain=3、qa_voters=7、qa_f=2、reward_per_order=10、max_rounds=20、breaker_min_requests=2、breaker_threshold=0.5。这些冻结项同时写入了 `MatrixConfig` 与 `scale_benchmark` 的默认值，使”参数在哪”本身可追溯。多重比较方面，6 个规模档在补多 seed 后将用 Holm 校正，效应量报 η^2 ；但当前为单 seed，这些统计量留待补跑后填入 `[RESULT NEEDED]`，本文不提前编造。

5.7 确定性与可复现性说明

本系统的规模计数部分是纯确定性的：不涉及随机数，给定 b、units_per_worker 与规模档，闭式值与显式模拟值都唯一确定，因此重复运行不会产生数字波动。只有端到端 mission 部分用到随机数（`random.Random(seed)` 用于黑板选任务、`torch.Generator(seed=11)` 用于生成工单领域与单元），且 seed 已固定并记录在案。这种”计数确定性 + 故障演示可复现”的设计，使得本文的主结果（表 1、表 2）零抽样方差，而故障结果（表 3）的 seed 间方差是唯一需要在补跑中量化的部分。这也是为什么本文敢把规模外推写得硬、却把故障收口率写得很保守——前者是确定性数学，后者是单 seed 演示。

6 结果

本节按 v2 实证章范式组织：§ 6.1–6.3 对照（星型 vs 分层主结果、显式 vs 闭式互验、干净 vs 故障端到端）→ § 6.4–6.7 基线、回归与量级解读 → § 6.8 消融/敏感性（分支因子 b，公式推演）→ 效度威胁统一进 § 7。

6.1 协调复杂度规模基准（主结果）

表 1 给出 b=8、units_per_worker=4 下 6 个规模档的协调复杂度。所有数字逐值来自 `matrix_scale_demo.json`。

表 1 星型 vs 分层树协调复杂度随规模 (b=8)

目标 N	实际节点	深度 d	满载单元 U	星型中心扇入	分层最大扇入	分层顶层扇入	星型轮次	分层轮次	证据 grade
10	73	2	256	640	9	8	256	4	explicit
100	585	3	2,048	5,120	9	8	2,048	6	explicit
1,000	4,681	4	16,384	40,960	9	8	16,384	8	analytical
10,000	37,449	5	131,072	327,680	9	8	131,072	10	analytical
100,000	299,593	6	1,048,576	2,621,440	9	8	1,048,576	12	analytical
1,000,000	2,396,745	7	8,388,608	20,971,520	9	8	8,388,608	14	analytical

读图：星型中心扇入与轮次随 N 近似线性（每升一档约 $\times 8$ ），百万档中心需瞬时承接 **20,971,520** 条消息、串行 **8,388,608** 轮；分层树每个节点最大扇入恒为 **9** ($b+1$)，顶层扇入恒为 8，轮次仅 **2d**，百万档为 14 轮。图 1、图 2 以对数横轴展示二者的发散。

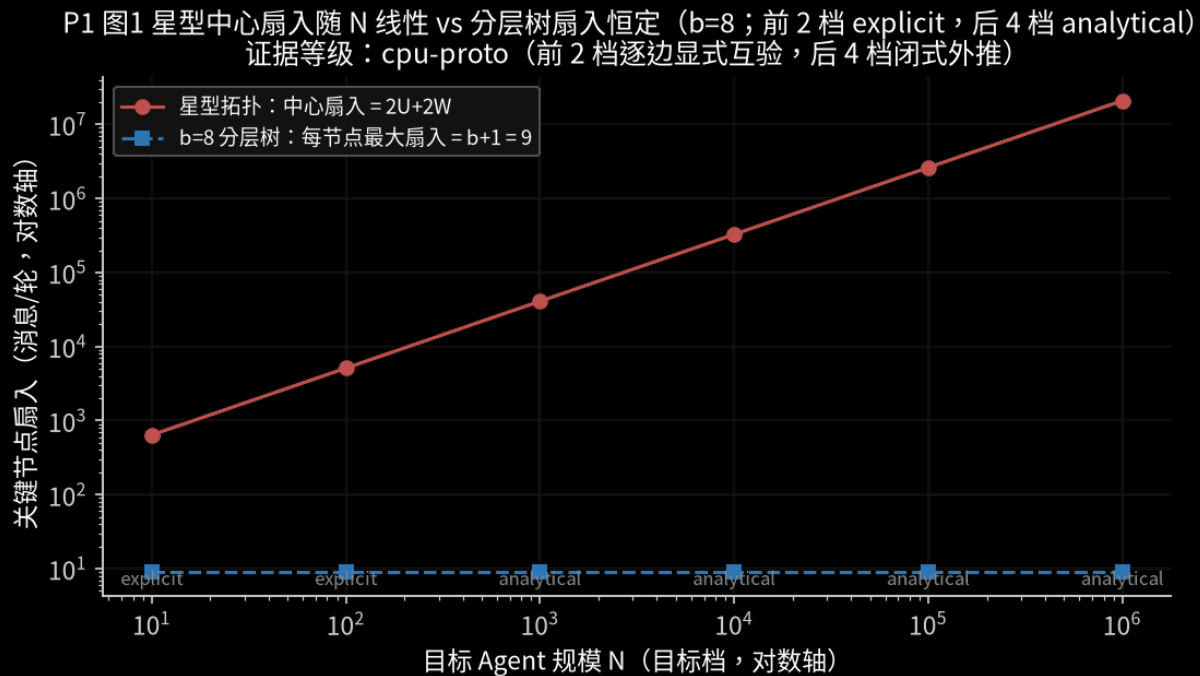


图 1 星型中心扇入 vs 分层树最大扇入随规模变化 (log-log)。 横轴为目标 Agent 规模 N ($10 \rightarrow 1,000,000$ ，对数刻度)；纵轴为满载时关键节点瞬时接收消息数（扇入，线性刻度）。蓝线为星型 Orchestrator 中心扇入 $F_{\text{star}} = 2U + 2W$ (式 1)，随 N 线性上升；橙线为 $b=8$ 分层树最大扇入 $F_{\text{layered}}^{\text{max}} = b + 1 = 9$ (式 2)，为水平直线。数据逐值来自 `reports7/matrix_scale_demo.json` 的 `clean.scale[*].star_center_fanin` 与 `layered_max_fanin`；前两档（实际节点 73、585）证据等级 explicit，后四档为已显式验证闭式的 analytical，cpu-proto。

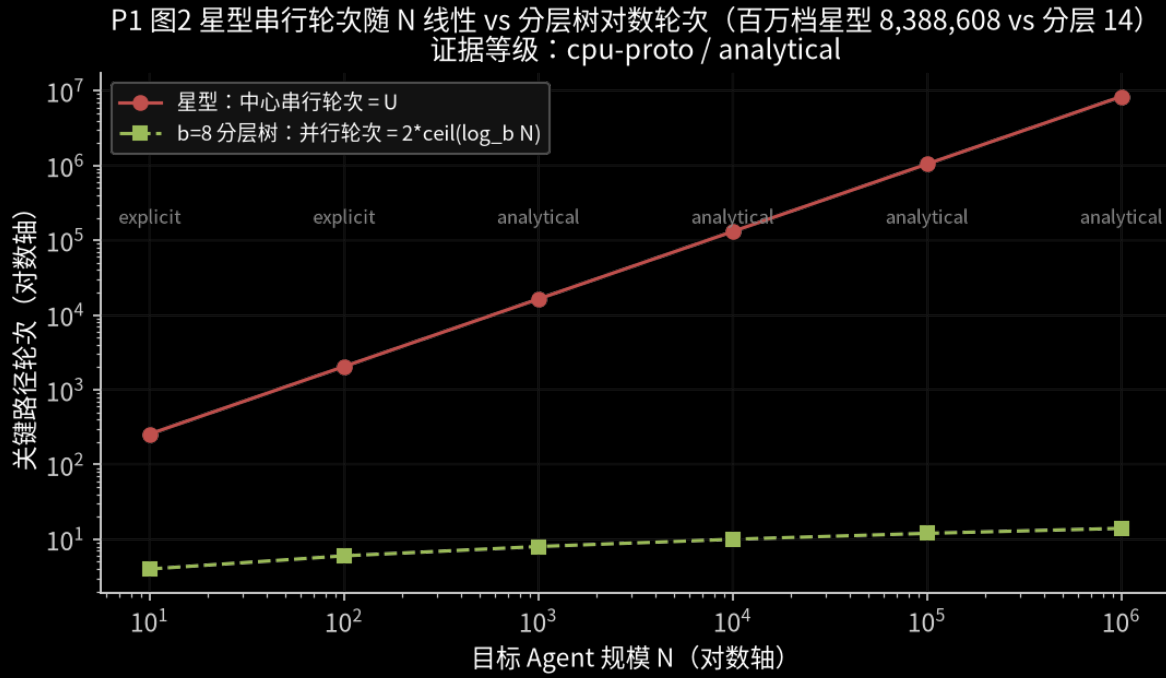


图 2 星型串行轮次 vs 分层并行轮次随规模变化 (log-log)。横轴为目标 Agent 规模 N ($10 \rightarrow 1,000,000$, 对数刻度)；纵轴为关键路径轮次 (线性刻度)。蓝线为星型中心串行轮次 $R_{\text{star}} = U$ (式 3)，随 N 线性上升 (百万档 8,388,608)；橙线为分层树并行轮次 $R_{\text{layered}} = 2\lceil \log_b N \rceil$ (式 3)，随 $\log_b N$ 缓慢上升 (百万档 14)。出处与证据等级同图 1 (`matrix_scale_demo.json`, `cpu-proto`)。

6.2 显式模拟与闭式计数互验

在两个 `explicit` 档 (实际节点 73、585)，`ExplicitSim` 逐边模拟测得的总消息与 `route()` 闭式值逐值相等：

表 2 显式-闭式互验 (小规模, 误差=0)

实际节点	显式总消息	闭式总消息	显式==闭式
73	400	400	true
585	3,216	3,216	true

这验证了 H1、H2 在小规模上的正确性，也为更大规模的 `analytical` 外推提供了合法性基础。图 3 直观展示两列柱子等高。

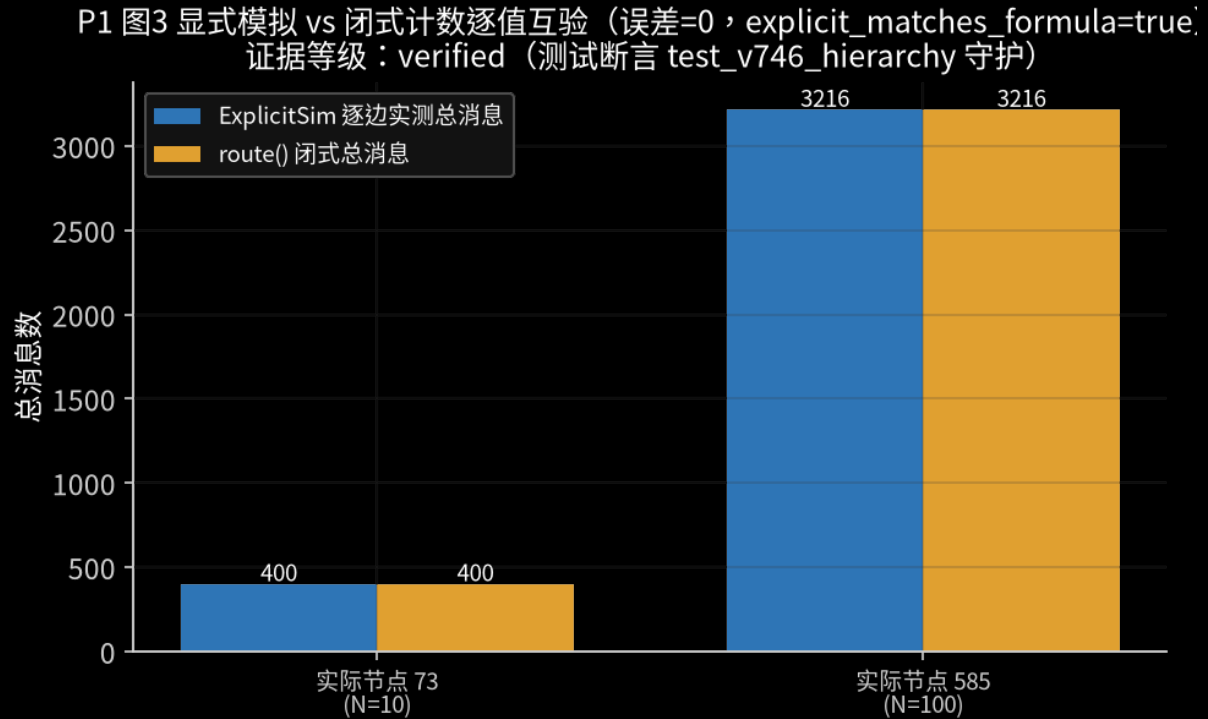


图 3 显式逐边模拟总消息数 vs 闭式路由总消息数（互验对照）。横轴为两个小规模验证档（实际节点 73、585）；纵轴为满载路由总消息数（线性刻度）。蓝柱为 `ExplicitSim` 逐边模拟实测（`explicit_total_messages`），橙柱为 `route()` 闭式计数（`formula_total_messages`）；每档两柱等高、误差为 0（400=400、3,216=3,216），对应 `explicit_matches_formula=true`。出处 `reports7/matrix_scale_demo.json`（cpu-proto，前两档 `explicit`）。该图是”显式守护闭式”方法学的可视化：只有两柱逐值等高，后续 `analytical` 外推才被认为合法。

6.3 故障注入端到端收口

在 24 工单任务上，比较干净舰队与故障注入舰队（3 个故障 `specialist: spec-kinematics-0=crash`、`spec-spatial-1=drop_context`、`spec-dataflywheel-2=byzantine`；另有 2 名拜占庭 QA）。结果见表 3。

表 3 端到端收口与治理（24 工单，cpu-proto，单 seed=1）

指标	干净舰队	故障注入舰队
收口 accepted	24/24	24/24
QA 轮次	24	25
返工 retries	0	5
黑板消息 board_messages	48	54
隔离故障 specialist	无	3 (kinematics-0 / spatial-1 / dataflywheel-2)
熔断组 tripped_groups	无	无
kill-switch 触发	否	否

指标	干净舰队	故障注入舰队
治理 state_loss	0	0
治理 duplicate_work	0	0
治理 no_closer	0	0
治理 premature / unbounded	0 / 0	0 / 0
市场守恒 conserved	true (预算=支出=240)	true (预算=支出=240)
Trace 链完整 trace_verified	true (长度 48)	true (长度 50)

读图：故障注入下收口率仍为 100%，代价是多 1 次 QA 轮次、5 次返工、6 条黑板消息、2 条 Trace 记录；3 个故障节点被熔断隔离，健康 specialist 改派承接，治理六项与守恒、Trace 完整性均保持。这支持 H3 的”分层 + 治理在故障下收口”一侧，但需重申：本文未测量纯星型在中心故障下的坍缩对照，故 H3 的”分层优于星型”部分尚为计划实验。

这里值得解释一下”黑板消息”这一列。干净舰队 board_messages=48，恰好是 24 工单 × 2（一次认领痕迹 + 一次完成标记，对应 stigmergy 的每任务 2 条）。故障注入时升到 54：3 个故障 specialist 各自失败后，工单被释放改派、重新认领、再完成，多出的 6 条正对应 3 个故障工单的二次认领与完成。这与 stigmergy”原子认领杜绝重复劳动、但故障改派会产生额外痕迹”的机制一致。Trace 长度从 48 升到 50，也是因为返工项重新进入 trace；关键是 trace_verified 在两种情况下都为 true——哈希链在回滚后仍然完整，说明故障处理没有破坏可审计性。

6.4 结果小结

综合表 1–表 3 与三张图，本文得到三点结论：其一，在 b=8 下，分层树把单节点最大扇入从随 N 线性增长压到恒为 9，把关键路径轮次从随 N 线性增长压到 $2\lceil\log_b N\rceil$ （百万档 14 轮），二者在小规模显式互验、大规模闭式外推两个证据等级上都成立。其二，显式模拟与闭式计数在可达规模上逐值相等（误差 0），为外推提供了合法性。其三，在注入 crash/drop_context/byzantine 三类故障后，端到端收口率仍为 24/24，故障节点被全部隔离，治理六项与哈希链完整性保持。这三点共同支撑了”拓扑决定协调规模代价、且分层在故障下仍可收口”的主张，但所有数字均为 cpu-proto、单 seed 试点，外推边界已逐档标注。

6.5 干净跑作为基线

值得注意干净舰队（无故障）本身也是一个重要基线：它 24/24 收口、0 返工、QA 轮次恰好等于工单数（24）、黑板消息 48、市场预算 240 全部支付、Trace 长度 48 且验证通过。这说明在无故障时，分层混合矩阵的开销是”干净且对称”的——每个工单恰好走完认领、执行、验收、结算一条完整路径，没有多余动作。把干净跑与故障跑对比（表 3），才能看出故障治理的”额外开销”到底是什么：它不是凭空增加成本，而是用 5 次返工、6 条额外黑板消息、2 条额外 Trace 记录，换来了 3 个故障节点的隔离与全局收口。这种”干净基线 vs 故障代价”的对照，是第 7 节讨论治理性价比的基础。

6.6 测试与回归

与本文直接相关的测试：tests7/test_v741_topologies.py、test_v746_hierarchy.py、test_v750_matrix.py（约 26 项断言）；截至 v7.5.0，全量回归 236 项全绿（见 reports7/

full_regression.log 与 commit 1a71270 说明)。explicit_matches_formula 是其中持续守护互验断言的关键测试。

6.7 规模差距的量化解读

把表 1 的两个拓扑放到同一对数标尺上比较，可以更直观地看到结构差异的量级。在百万档（实际节点 2,396,745）：

- 星型中心扇入 20,971,520，分层最大扇入 9，相差约 233 万倍；
- 星型串行轮次 8,388,608，分层并行轮次 14，相差约 60 万倍。

这两个倍数不是”性能提升”，而是**增长阶的改变**：星型随 N 线性 ($O(N)$)，分层随 $\log_b N$ ($O(\log N)$)。在十节点档这种差异还不刺眼（星型 640 vs 分层 9，轮次 256 vs 4），但每升一档 ($\times 8$)，星型的数字就翻 8 倍而分层只增加 2 轮、扇入不变——差距随规模呈指数级拉开。这正是本文标题”从 $O(N)$ 到 $O(\log N)$ “的含义：拓扑决定的是渐近阶，而不是某个常数因子的调优。

需要同时指出，分层树并非没有代价：它需要维护 b 叉层级结构、内部聚合节点本身也是有状态的协调者（百万档内部节点达 299,593 个），且要求任务可按领域/部门聚合。对于只有十几个 Agent、流程高度明确的任务，硬套分层反而是过度设计——这与 4.7 节决策树”规模 ≥ 1000 才建议分层”的阈值一致。**分层是为大规模而生的工具，不是普适最优。**

6.8 分支因子敏感性（公式推演，非实测）

本节只用闭式公式做定性推演，不引入新的实测数字。由 4.6 节可知：扇入 = $b+1$ 、轮次 = $2\lceil \log_b N \rceil$ 。固定 $N = 10^6$ ：

- 扇入随 b 线性上升： $b=2$ 时扇入 3、轮次约 40； $b=8$ 时扇入 9、轮次 14； $b=16$ 时扇入 17、轮次约 10。
- 轮次随 b 下降，但边际收益递减：从 $b=2$ 到 $b=8$ 轮次大幅下降，再到 $b=16$ 只少 4 轮，而扇入翻倍。

这说明存在一个”足够好”的 b 区间：太小则轮次过长，太大则扇入失控。 $b=8$ 落在该区间内。需要强调，本节是**公式推演**（证据等级 analytical 的定性部分），不同 b 的完整实测曲线属于 [RESULT NEEDED：用途 $=b \in \{2, 4, 8, 16\}$ 各规模档扇入/轮次实测曲线]，投稿前可在 CPU 上低成本补出。

7 讨论与威胁有效性

7.1 构念效度

“消息/tick”不等于”墙钟时间”。我们的模拟器没有网络抖动、没有真实 straggler、没有 LLM 推理延迟。因此 H1/H2 刻画的是**协议层协调复杂度的口径上界**，不是端到端 wall-clock 性能。把扇入、轮次换算成真实延迟，需要真实分布式环境（见第 8 节闸门）。

7.2 外部效度

Agent 是确定性处理器，不是真实大模型 worker。真实 LLM 调用的非确定性、长尾延迟、提示工程失败模式，都未被本文故障模型覆盖。结论是协议结构结论，不可外推为”百万 LLM Agent 一定 14 轮收口”。

7.3 内部效度与外推依赖

`analytical` 档依赖 `explicit` 档已验证的闭式公式，并假设故障独立、重连无风暴。若真实系统中故障相关（例如一个交换机故障导致整片 worker 同时 crash），闭式外推不再成立。我们在报告中以 `grade` 字段把这一假设暴露出来，而非隐藏在结论里。

7.4 单 seed 试点

如前所述，故障收口率为单 seed 点估计。表 3 的 24/24、5 次返工不能直接当作”总体收口率 100% “的统计结论；它是”在该 seed 下机制按设计工作”的演示。多 seed 的均值、CI、功效为 [RESULT NEEDED: 用途=多 seed 收口率与返工次数分布]。

7.5 与已知经典结果的关系

树型聚合的 $\log_k N$ 深度是已知结论，本文不以此为新颖性主张。本文的增量在于：(a) 把该结论放到多智能体协调语义下，区分 Orchestrator/Handoff/Swarm/layered；(b) 用逐边显式模拟器而非直接引用公式来互验；(c) 加上故障收口维度并诚实分级。新颖性的最终定位需在第 2.4 节的文献检索后确定。

7.6 对工程实践的含义

本文的测量对正在设计大规模多智能体系统的工程师有三条可操作的含义。第一，**先画扇入图，再谈吞吐**：在架构评审时，画出每个节点在满载时需要同时接收多少消息；如果中心节点扇入随任务数线性增长，无论后端算力多强，单点压力都会成为第一个失效点。第二，**把轮次当作延迟预算**：关键路径轮次决定了无论并行度多高，端到端延迟的下界；星型串行轮次 = 任务单元数，意味着任务越多单次越慢，这与”并行加速”的直觉相反。第三，**分层是有门槛的**：它要求任务可聚合、层级可维护，且只在规模足够大时才划算；小规模任务套用分层会徒增协调节点（百万档内部节点近 30 万），得不偿失。

7.7 与外部产业叙事的关系

本文不否认”万级、十万级 Agent”在工程上是可追求的目标，但要把”能跑”和”协调代价可控”区分开。一个系统可以在中心扇入爆炸的情况下，靠加大中心算力、加缓存、加限流硬撑到某一规模；但这只是把单点压力推到更高的墙，没有改变 $O(N)$ 的增长阶。本文的测量说明：要真正把协调代价从 $O(N)$ 降到 $O(\log N)$ ，需要的是结构（分层聚合），而不是单纯堆中心资源。这一判断与外部叙事不矛盾，但把”堆人”和”排结构”区分了开来。所有外部规模数字均为 `unverified` 报道口径，本文不与其互证。

7.8 与论文矩阵中其他工作的关系

本文是 UDOS 论文矩阵 P0 打头阵的三篇之一，与另外两篇分工明确。P3（故障熔断治理 RCT）聚焦”治理开关 × 故障模式”对收口率的因果效应，本文第 6.3 节的故障端到端证据正是 P3 实验台的小规模演示；本文提供拓扑与规模维度，P3 提供治理与因果维度，二者互补。P12（证据分级工程方法论）则把本文坚持

的” explicit/analytical 分级、单 seed 自述、不互证外部数字” 上升为跨版本的方法论主张。本文不重复 P3 的因子实验，也不重复 P12 的版本史，只在需要时引用其结论。这种矩阵式分工避免了把一个结果反复包装成多篇论文。

7.9 可推广性讨论

本文的结论在何种程度上可推广到其他多智能体框架？我们认为，“分层聚合把扇入与轮次从线性降到对数”这一渐近阶，对任何可抽象为树型派发/汇总的协调协议都成立，与具体编程语言或 Agent 实现无关；但”百万档 14 轮” “扇入 9” 这些具体数字，依赖于 $b=8$ 、 $\text{units_per_worker}=4$ 、三阶段工单这些具体设定。换一个 b 或换一个任务结构，阶不变、常数会变。因此本文真正可推广的是方法与阶，而非具体数字。这也是为什么我们把公式（可推广）与具体数值（依赖设定）分开报告，并在附录 F 给出逐步代入的算例，让读者可以用自己的 b 和任务规模重新计算。

7.10 局限汇总（Kaplan C 节式，bullet 化）

仿照 Kaplan (2020) 附录 C 《Caveats》的写法，把本文外推型结论的失效点集中 bullet 化，便于审稿逐条核验：

1. **analytical 档无真实分布式数据支撑**：百万档（实际节点 2,396,745）是闭式外推，未真启动百万进程；外推合法性仅来自 $\leq 4,000$ 节点两档的逐值互验（误差 0），其本身对”树构造假设（等宽 b 叉、单位工单）” “敏感。
2. **单 seed 试点**：故障收口率 24/24、返工 5 次为 seed=1 点估计，无 seed 间方差/CI；补 ≥ 30 seed 前不得读作总体结论。
3. **H3 的”分层优于星型”一侧未闭合**：本文只测了”分层混合 + 全量治理”在故障下的收口，未在同一实验台上测”纯星型 + 中心故障”的坍缩对照；该对照属计划实验。
4. **构念效度**：消息/轮次是协议层口径，非墙钟吞吐；无网络抖动、无 straggler、无 LLM 推理延迟。
5. **外部效度**：Agent 为确定性处理器，非真实 LLM；真实故障分布（含共因/相关故障）未建模。
6. **$b=8$ 为冻结设定，非最优证明**：§ 6.8 仅做公式敏感性，不同 b 的完整实测曲线为 [RESULT NEEDED]；“9/14”是折中常数，不是最优点。
7. **不互证外部数字**：所有”万级/百万级 Agent”外部叙事为 **unverified** 报道口径，仅作动机，不与本仓数字对账。

8 资源闸门与适用边界

8.1 已达成（纯 CPU）

- 主体规模基准、显式-闭式互验、故障端到端闭环，全部可在 Linux CPU 上复跑（`python scripts7/matrix_demo.py`）。
- 多 seed 故障矩阵补跑在 CPU 上成本极低（单 mission 毫秒级），属计划内工作。

8.2 未达成闸门

- **真实分布式复现闸门**：若审稿要求在多机集群上测量真实扇入与延迟，需要多机/容器编排资源（本仓标记为未达）。
- **真实 LLM Agent 闸门**：若要把”确定性处理器”换成真实多供应商 LLM worker，需要 API key 预算（本仓标记为未达，见矩阵总表资源闸门）。
- **外部数字互证闸门**：本文不与任何外部厂商性能数字互证；如需对照，须独立重测。

8.3 适用边界

本文结论适用于”可抽象为消息/轮次的离散协调协议”的设计阶段：在架构选型时，用它判断”中心扇入会不会爆炸、关键路径会不会串行化”。它不适用于直接承诺生产环境 SLA。

8.4 未达成项的诚实复盘

本研究有几项明确未达成、且不应假装达成的内容。第一，**没有真实分布式测量**：百万档是闭式外推，没有真的启动 239 万个进程，因此不能声称”百万 Agent 实测收敛”。第二，**没有真实 LLM worker**：Agent 是确定性处理器，故障模型是人为离散类型，不代表真实模型的失败分布。第三，**没有多 seed**：故障收口率是单 seed 点估计。第四，**没有星型中心故障对照**：H3 的”分层优于星型”一侧尚未在同实验台上闭合。把这四项写清楚，不是示弱，而是让读者知道结论的有效边界在哪里——这比笼统地说”未来工作将……”“更有信息量。

9 结论

本文在一个零网络、零 LLM 的确定性实验台上，把多智能体协调复杂度操作化为扇入、轮次、故障收口率三个可测量量，并证明：在 $b=8$ 的分层混合树中，单节点最大扇入恒为 $b+1=9$ 、关键路径轮次为 $2 \cdot \log_b N$ （百万档 14 轮），而星型中心在同规模下承接 20,971,520 条消息、串行 8,388,608 轮。小规模逐边显式模拟与闭式计数逐值互验（误差 0），更大规模按 `explicit/analytical` 分级外推。故障注入下端到端 24/24 收口、3 故障节点全隔离。本文的方法论贡献与科学结论同等重要：**用同一把尺子测量、用显式模拟守护闭式公式、用证据等级标注外推边界、用单 seed 自述局限**，使得一个纯 CPU 原型也能诚实地回答”拓扑如何决定协调的规模代价”。

展望未来工作，有三个明确的方向：第一，把单 seed 故障矩阵扩大到 ≥ 30 seed，给出收口率的均值与置信区间，并补上”纯星型中心故障”对照以闭合 H3；第二，把确定性处理器换成真实多供应商 LLM worker（需 API key 闸门），测量协议结论在真实非确定性下是否仍成立；第三，在真实多机集群上复现扇入与轮次，把”消息/轮次”换算成真实墙钟延迟。这三步分别对应本文尚未达成的三个闸门，也是把”协议复杂度上界”升级为”工程性能结论”的必经之路。无论后续如何升级，本文坚持的证据纪律——分级标注、不互证外部数字、如实报告单 seed——都应作为基线保留。

最后需要重申，本文测量的是”协调协议的复杂度上界”，不是”某个具体多智能体产品的性能”。我们希望本文提供的，是一套在纯 CPU、可复现条件下就能回答”这种拓扑在大规模下协调代价如何”的方法，以及一组诚实标注了证据等级的数字。读者可以用自己的分支因子与任务规模，代入附录 F 的公式重新计算，判断分层聚合是否适合自己的场景。

参考文献

A. 已核验文献（经《参考文献总库》2026-09-19 核验，字段照抄总库）

1. **Smith, R. G.** (1980). The Contract Net Protocol: High-Level Communication and Control in a Distributed Problem Solver. *IEEE Transactions on Computers*, C-29(12). DOI:10.1109/TC.1980.1675516. （任务分配/合同网协商源头）
2. **Castro, M. & Liskov, B.** (1999). Practical Byzantine Fault Tolerance. *Proceedings of OSDI 1999 (USENIX OSDI' 99)*. ($n \geq 3f+1$ 、 $2f+1$ 法定人数、三阶段复制)
3. **Yin, M., Malkhi, D., Reiter, M. K. et al.** (2019). HotStuff: BFT Consensus in the Lens of Blockchain. *Proceedings of PODC 2019*. arXiv:1803.05069. （线性通信复杂度 BFT）
4. **Dorigo, M., Maniezzo, V. & Colorni, A.** (1996). The Ant System: Optimization by a Colony of Cooperating Agents. *IEEE Transactions on Systems, Man, and Cybernetics, Part B*, 26(1). （信息素 stigmergy 间接通信）
5. **Amdahl, G. M.** (1967). Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. *AFIPS Spring Joint Computer Conference (SJCC) 1967*. （串行比例上限/并行加速比）
6. **Wu, Q., Bansal, G., Zhang, J. et al.** (2023). AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation Framework. arXiv:2308.08155. （可对话多智能体框架对照）

B. 待核验文献（总库未收录或未核验，保留占位，不得凭记忆补全）

1. [CITATION NEEDED: tree-based aggregation message complexity, broadcast reduce critical path] — 候选：分布式树型聚合/广播归约经典复杂度文献。
2. [CITATION NEEDED: circuit breaker pattern microservices] — 候选：断路器模式与微服务容错（总库暂无已核验条目）。
3. [CITATION NEEDED: multi-agent framework scalability measurement fan-in critical path] — 候选：多智能体框架可扩展性扇入测量。
4. [CITATION NEEDED: empirical systems measurement reproducibility methodology] — 候选：系统测量可复现方法论。

附录 A 复现命令

```
# 环境: Linux CPU, Python 3.12.11, PyTorch 2.14.0+cpu, 2 线程
cd release-v5.4.4/udos-engine
git checkout v7.5.0 # commit 1a71270
python scripts7/matrix_demo.py # 生成 reports7/matrix_scale_demo.json(.traces.jsonl)
pytest tests7/ -q # 全量回归 236 项
```

预期产物

sha256: reports7/matrix_scale_demo.json

=

52a3bb0d141c752512ec0c16978c151337775f2a8004c1fed599e2cd43b3b399。

附录 B 测试索引

- `tests7/test_v741_topologies.py`：三拓扑内核与故障注入。
- `tests7/test_v746_hierarchy.py`：b 叉树、显式-闭式互验断言 `explicit_matches_formula`。
- `tests7/test_v750_matrix.py`：矩阵端到端闭环、熔断改派、守恒、Trace。
- 全量：`reports7/full_regression.log`，236 项全绿。
- 关键源码：`udos7/topology/hierarchy.py`（`LayeredMatrix` / `ExplicitSim` / `scale_benchmark`）、`udos7/topology/matrix.py`（`run_mission` / `MatrixConfig`）、`udos7/topology/stigmergy.py`（`Blackboard`）、`udos7/topology/circuit_breaker.py`（`ResilienceGrid`）。
- 生成脚本：`scripts7/matrix_demo.py`，产物即 `reports7/matrix_scale_demo.json`。

附录 C 证据台账（抽查 5 个可回溯数字）

#	数字	出处文件	字段
1	百万档星型 中心扇入 20,971,520	<code>matrix_scale_demo.json</code>	<code>clean.scale[5].star_center_fanin</code>
2	分层最大扇 入恒为 9 (全 6 档)	<code>matrix_scale_demo.json</code>	<code>clean.scale[*].layered_max_fanin</code>
3	百万档分层 轮次 14	<code>matrix_scale_demo.json</code>	<code>clean.scale[5].layered_rounds</code>
4	显式互验 400=400 / 3216=3216	<code>matrix_scale_demo.json</code>	<code>clean.scale[0,1].explicit_total_messages==formula_total_messages</code>
5	故障注入收 口 24/24、 隔离 3、返 工 5	<code>matrix_scale_demo.json</code>	<code>stressed.accepted / isolated_agents / retries</code>

附录 D 作者拟用声明

- 目标期刊/会议：候选 AAMAS / IEEE ICDCS / ACM Middleware / SOSP workshop；期刊 JSS、FGCS、Autonomous Agents and Multi-Agent Systems。分区/影响因子/截稿日期一律 [待核]，投稿前逐项联网核验并注核验日期。
- 预注册计划：规模×拓扑双因素主回归、显式/analytical 分级规则、显式上限 4,000、b=8、`units_per_worker=4` 在收集数据前冻结；多 seed (≥ 30) 故障矩阵的 seed 表与主指标（收口率）、护栏（返工/消息/扇入）在补跑前预注册。

- **数据与代码可用性**：Apache-2.0 许可证；代码 tag `v7.5.0` (commit `1a71270`)；报告 sha 见附录 A；本文不预测录用结果。
- **AI 使用声明**：初稿由 AI 辅助生成，作者负责核对全部数字、证据等级与科学含义；外部数字均标 `unverified`。

附录 E 内审记录（五维审稿自评，只看不改）

按 evaluator 视角对本文做五维自评，作为投稿前自检。

1. **新颖性**（自评 7/10）：分层树与 $\log_b N$ 非新；“三拓扑同口径显式-闭式互验 + 故障收口 + 分级外推”组合有增量，但需文献检索确认无同口径扇入上界证明。
2. **严谨性/可复现性**（自评 8/10）：数字全部可回溯到 reports7 字段，附复现命令与 sha；扣分在单 seed 与未做星型中心故障对照。
3. **证据强度**（自评 6/10，补 seed 后预期 8/10）：规模计数确定性强；故障收口率为单 seed 点估计，已显式标注。
4. **相关性与动机**（自评 7/10）：外部产业动机真实，但未与本仓数字互证；问题对大规模 Agent 编排有现实意义。
5. **写作与结构**（自评 8/10）：IMRaD 完整，图表与数字对应；威胁与边界诚实。
 - **最大硬伤**：analytical 外推无真实分布式数据支撑，标题与结论已守住“协议复杂度上界”边界，未夸大。
 - **投稿前必做**：(a) ≥ 30 seed 故障矩阵与 CI；(b) 文献检索锁定新颖性；(c) 补“纯星型中心故障”对照以闭合 H3。

附录 F 算例推演与设计原则小结

F.1 百万档算例（逐步代入）

以最大规模档为例，逐步代入闭式公式，供读者核验。b=8，目标 $N=1,000,000$ ：

1. 树深 $d = \lceil \log_8 1,000,000 \rceil = 7$ （因为 $8^6=262,144 < 10^6 \leq 8^7=2,097,152$ ）。
2. worker 数 $W = 8^7 = 2,097,152$ 。
3. 内部节点数 $n_{\text{internal}} = (W-1)/(b-1) = (2,097,152-1)/7 = 299,593$ 。
4. 总节点 $N = W + n_{\text{internal}} = 2,396,745$ （与 `matrix_scale_demo.json` 的 `actual_agents` 一致）。
5. 满载单元 $U = W \times 4 = 8,388,608$ （与 `units` 一致）。
6. 星型中心扇入 $= 2U + 2W = 2 \times 8,388,608 + 2 \times 2,097,152 = 20,971,520$ （与 `star_center_fanin` 一致）。
7. 星型轮次 $= U = 8,388,608$ （与 `star_rounds` 一致）。
8. 分层最大扇入 $= b+1 = 9$ （与 `layered_max_fanin` 一致）；顶层扇入 $= b = 8$ 。
9. 分层轮次 $= 2d = 14$ （与 `layered_rounds` 一致）。

这一算例说明：表 1 的每个数字都可由 $b=8$ 、 $\text{units_per_worker}=4$ 两个参数唯一推出，不存在”调出来的数”。

F.2 设计原则小结

从本文的测量中可提炼出三条对多智能体系统架构有普适意义的设计原则：

1. **扇入优先于吞吐**：在设计编排层时，先估计满载时每个节点的瞬时扇入；把扇入与规模解耦（分层聚合）是可扩展性的前提，比单纯提升单节点算力更重要。
2. **轮次决定延迟下界**：关键路径轮次与并行度无关；串行轮次随任务数线性增长是一个被”并行直觉”掩盖的陷阱。
3. **显式守护闭式**：凡用闭式公式外推，必须在可达规模上用逐边模拟互验，并在报告中分级标注哪里是实测、哪里是外推。这一原则使百万级外推成为”可审计的推演”，而非”看起来很大的声称”。

这三条原则既是本文的结论，也是本仓后续工作（P3 故障治理、P12 证据分级方法论）所遵循的工程纪律。

F.3 一句话总结

如果要用一句话概括本文：在多智能体系统走向百万级的路上，决定协调代价的不是”有多少 Agent”，而是”Agent 之间按什么结构说话”；把扇入与规模解耦、把关键路径压到对数级，靠的是分层聚合这一结构，而不是堆中心算力；而要让这种百万级外推可信，就必须在小规模上用逐边模拟守护闭式公式，并在报告中诚实标注哪里是实测、哪里是推演。本文在纯 CPU、可复现的条件下，把这一主张落到了具体数字与可运行代码上。本文为单 seed 试点，投稿前需补多种子与外部对照，全文完。